These are the 10 test cases I done from the 100 case studies

CASE STUDY 1(number:12)

1. aim: Attendance Recovery Computation.py

2. objective:

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under different future attendance scenarios.

3. python code:

```
attended=int(input("enter classes attended: "))
conducted=int(input("enter classes conducted: "))
print("required 75% attendance")
print(f'your attendance is {100*attended/conducted:.2f} ')
to_attend=0
present=100*attended//conducted
if present<75:
    while present<75:
        to_attend=to_attend+1
        present=((to_attend+attended)/(conducted+to_attend))*100
    print(f"you need to attend {to_attend} upcoming classes to aquire required
attendance")
else:
    print(f'you have required attendance')
```

4.test cases and outputs:

```
enter classes attended: 14
enter classes conducted: 20
required 75% attendance
your attendance is 70.00
you need to attend 4 upcoming classes to aquire required attendance
```

CASE STUDY 2(number:3)

1. aim: Hospital Billing and Patient Classification.py

2. objective:

Process consultation, diagnostic, and treatment charges, apply conditional concessions, validate billing constraints, and classify patients according to a supplied payment policy.

3. python code:

```
patients = []
n = int(input("Enter number of patients: "))
for i in range(n):
    print("Enter details for Patient",i+1)
```

```
    name = input("Enter patient name: ")
    consultation = float(input("Enter consultation charges: "))
    diagnostic = float(input("Enter diagnostic charges: "))
    treatment = float(input("Enter treatment charges: "))
    total = consultation + diagnostic + treatment
    if total>=10000:
        discount=(total/100)*20
    elif total>=5000:
        discount=(total/100)*10
    else:
        discount = 0
    total_bill = total - discount
    if total_bill >= 10000:
        category = "High Billing"
    elif total_bill >= 5000:
        category = "Medium Billing"
    else:
        category = "Low Billing"
    patients.append([name, total, discount, total_bill, category])
for i in patients:
    print("Patient Name:", i[0])
    print("Total Charges: Rs.", i[1])
    print("Concession: Rs.", i[2])
    print("Final Bill: Rs.", i[3])
    print("Classification:", i[4])
print("Total patients processed:", len(patients))
```

4. test cases and outputs:

```
Enter number of patients: 2
Enter details for Patient 1
Enter patient name: mustafavali
Enter consultation charges: 1000
Enter diagnostic charges: 2000
Enter treatment charges: 3000
Enter details for Patient 2
Enter patient name: karthik
Enter consultation charges: 2000
Enter diagnostic charges: 1000
Enter treatment charges: 3500
Patient Name: mustafavali
Total Charges: Rs. 6000.0
Concession: Rs. 600.0
Final Bill: Rs. 5400.0
Classification: Medium Billing
Patient Name: karthik
Total Charges: Rs. 6500.0
Concession: Rs. 650.0
Final Bill: Rs. 5850.0
Classification: Medium Billing
Total patients processed: 2
```

CASE STUDY 3(number:58)

1. aim: Library Borrowing and Fine Reconciliation.py

2. objective:

Analyse tuples representing book identifiers, borrowing periods, and return delays; calculate fines, identify overdue records, and reconcile book availability.

3. python code:

```
valid_books=["book1","book2","book3","book4","book5"]
books=['book1'*3,'book2'*3,'book3'*3,'book4'*3,'book5'*3]
print("available actions: take , return")
request=input("what is your request: ")
if request=="take":
    name=input("what book do you want: ")
    if name in books:
        books.remove(name)
        print("return within 5 days!")
    else:
        print("this is not available")
elif request=="return":
    name=input("what book are you returning: ")
    if name in valid_books:
        books.append(name)
    time=int(input("for how many days did you take this book?: "))
    if time>5:
```

```
        fine=50
        for i in range(5,time):
            fine=fine+50
    else:
        fine=0
    print(f"you need to pay {fine}")
```

4. test cases and outputs:

```
available actions: take , return
what is your request: return
what book are you returning: book1
for how many days did you take this book?: 7
you need to pay 150
```

CASE STUDY 4(number:58)

1. aim: Multi-Product Shopping Budget Constraint.py

2. objective:

Evaluate a shopping basket containing multiple products and quantities, apply discount conditions, and determine which purchases must be reduced to remain within a fixed budget.

3. python code:

```
products=[]
total=0
budget=float(input("Enter your shopping budget: Rs."))
n=int(input("Enter number of products: "))
for i in range(n):
    name=input("Enter product name: ")
    price=float(input("Enter price: ₹"))
    quantity=int(input("Enter quantity: "))
    cost=price*quantity
    total=total+cost
    products.append([name, price, quantity, cost])
for j in products:
    print(j[0], "- Rs.", j[1], "x", j[2], "= Rs.", j[3])
print("Total bill: Rs.", total)
if total <= budget:
    print("Shopping is within your budget.")
else:
    print("Shopping exceeds your budget.")
    print("Amount exceeded: ₹", total - budget)
```

4. test cases and outputs:

```
Enter your shopping budget: Rs.10000
Enter number of products: 5
Enter product name: shirt
Enter price: ₹500
Enter quantity: 4
Enter product name: pants
Enter price: ₹700
Enter quantity: 3
Enter product name: bags
Enter price: ₹1400
Enter quantity: 2
Enter product name: shoe
Enter price: ₹600
Enter quantity: 1
Enter product name: watch
Enter price: ₹1000
Enter quantity: 1
shirt - Rs. 500.0 x 4 = Rs. 2000.0
pants - Rs. 700.0 x 3 = Rs. 2100.0
bags - Rs. 1400.0 x 2 = Rs. 2800.0
shoe - Rs. 600.0 x 1 = Rs. 600.0
watch - Rs. 1000.0 x 1 = Rs. 1000.0
Total bill: Rs. 8500.0
Shopping is within your budget.
```

CASE STUDY 5(number:70)

1. aim: Multi-Stage Scholarship Shortlisting.py

2. objective:

Apply minimum academic eligibility, attendance requirements, income conditions, and ranking rules to select candidates within a fixed scholarship quota.

3. python code:

```
details=[]
details.append(input("Enter your name: "))
details.append(int(input("Enter your marks percentage: ")))
details.append(int(input("Enter your attendance percentage: ")))
details.append(int(input("Enter your income: ")))
if details[1]>60 and details[2]>70:
        if details[3]>10000:
            category='Merit-cum-Means scholarship'
        else:
            category='Merit-Based scholarship'
else:
    if details[3]>10000:
        category='Need-Based scholarship'
    else:
```

```
        category='no category'
print(f'the student can apply for scholarship under {category}')
```

4. test cases and outputs:

```
Enter your name: mustafavali
Enter your marks percentage: 80
Enter your attendance percentage: 70
Enter your income: 70000
the student can apply for scholarship under Need-Based scholarship
```

CASE STUDY 6(number:29)

1. aim: Password Policy Validation Engine.py

2. objective:

Evaluate sample password strings against minimum length, character-category, repetition, and prohibited-substring rules; explain each failed condition.

3. python code:

```
password1=0
while password1!="strong":
    up=0
    low=0
    spe=0
    num=0
    length=0
    password=input("enter password: ")
    for i in password:
        if 48<=ord(i)<=57:
            num+=1
        elif 65<=ord(i)<=90:
            up+=1
        elif 97<=ord(i)<=122:
            low+=1
        else:
            spe+=1
        length+=1
    if length>8:
        if length<20:
            if num>0:
                if low>0:
                    if up>0:
                        if spe>0:
                            print("your password is strong")
                            password1="strong"
                        else:
                            print("password doesn't have special characters")
                            password1="not strong"
                    else:
                        print("password doesn't have upper characters")
```

```
                    password1="not strong"
                else:
                    print("password doesn't have lower characters")
                    password1="not strong"
            else:
                print("password doesn't have numbers")
                password1="not strong"
        else:
            print("password id long")
            password1="not strong"
    else:
        print("password is short")
        password1="not strong"
```

4. test cases and outputs:

```
enter password: Asdf2322
password is short
enter password: A@#$adjdns
password doesn't have numbers
enter password: Ajefsj234@!
your password is strong
```

CASE STUDY 7(number:2)

1. aim: Progressive Electricity Billing.py

2. objective:

Calculate household bills using multiple consumption slabs, fixed charges, surcharges, and rebates, then compare bills across several months and identify unusual increases.

3. python code:

```
n = int(input("Enter number of households: "))
bills = []
for i in range(n):
    print("Household", i + 1)
    name = input("Enter household name: ")
    units = float(input("Enter electricity units consumed: "))
    if units <= 100:
        bill=units*2
    elif units<=200:
        bill= units*3
    elif units<=300:
        bill=units*5
    else:
        bill=units*7
    fixed_charge=50
    total_bill=bill+fixed_charge
    bills.append([name, units, total_bill])
for i in bills:
    print("Household:", i[0])
```

```
    print("Units consumed:", i[1])
    print("Energy charge: Rs.", i[2] - 50)
    print("Fixed charge: Rs. 50")
    print("Total bill: Rs.", i[2])
highest=bills[0]
for j in bills:
    if j[2]>highest[2]:
        highest=j
print("Highest bill:", highest[0])
print("Amount: Rs.", highest[2])
```

4. test cases and outputs:

```
Enter number of households: 2
Household 1
Enter household name: mustafavali
Enter electricity units consumed: 223
Household 2
Enter household name: srinivas
Enter electricity units consumed: 305
Household: mustafavali
Units consumed: 223.0
Energy charge: Rs. 1115.0
Fixed charge: Rs. 50
Total bill: Rs. 1165.0
Household: srinivas
Units consumed: 305.0
Energy charge: Rs. 2135.0
Fixed charge: Rs. 50
Total bill: Rs. 2185.0
Highest bill: srinivas
Amount: Rs. 2185.0
```

CASE STUDY 8(number:57)

1. aim: Smart Canteen Order and Stock Management.py

2. objective:

Process successive customer orders, calculate bills, update available inventory, reject unavailable quantities, and summarize daily sales and remaining stock.

3. python code:

```
rice=20
eggs=30
noodles=10
chicken=2000
items=['fried rice','noodles','chicken rice']
special=['eggs','chicken']
print("available items are: ")
for i in items:
```

```
    print(i)
print("additional:")
for i in special:
    print(i)
nxt_order=1
while nxt_order!="no":
    if rice<1 or eggs<1 or noodles<1 or chicken<200:
        nxt_order=0
        print("out of stock")
        break
    order=input("what you want to order: ").lower()
    extra=input("specialization?(0 if no need): ").lower()
    if order=="fried rice":
        rice=rice-1
        if extra=="egg":
            eggs=eggs-1
        elif extra=="chicken":
            chicken=chicken-200
    elif order=="chicken rice":
        rice=rice-1
        if extra=="egg":
            eggs=eggs-1
    elif order=="noodles":
        noodles=noodles-1
        if extra=="egg":
            eggs=eggs-1
        elif extra=="chicken":
            chicken=chicken-200
    else:
        print("there no such type of item")
    nxt_order=input("do you want next order('no' if dont need): ")
```

4. test cases and outputs:

```
available items are:
fried rice
noodles
chicken rice
additional:
eggs
chicken
what you want to order: fried rice
specialization?(0 if no need): chicken
do you want next order('no' if dont need): yes
what you want to order: noodles
specialization?(0 if no need): egg
do you want next order('no' if dont need): no
```

CASE STUDY 9(number:15)

1. aim: Smart Parking Fee Management.py

2. objective:

Calculate charges from entry and exit times, vehicle categories, overnight parking, daily limits, and penalty conditions while validating time inputs.

3. python code:

```
A=["car",'bike','truck','bicycle']
print(f'available slots for vehicles {A}')
parking_vehicle=input("enter your parking vehicle: ").lower().strip()
time=int(input("enter how many hours(0 if less than 1 hour): "))
if parking_vehicle =='car':
    maximum=10
    fee=100
elif parking_vehicle =='bike':
    maximum=5
    fee=50
elif parking_vehicle =='truck':
    maximum=15
    fee=200
else:
    maximum=0
    fee=0
if time>maximum:
    print(f'you need to pay extra 1000')
else:
    for i in range(time):
        fee=fee+20*(fee/100)
print(f'your total fee is {int(fee)}')
```

4. test cases and outputs:

```
available slots for vehicles ['car', 'bike', 'truck', 'bicycle']
enter your parking vehicle: car
enter how many hours(0 if less than 1 hour): 5
your total fee is 248
```

CASE STUDY 10(number:1)

1. aim: University Scholarship Allocation.py

2. objective:

Design a system that evaluates applicants using aggregate marks, subject-wise minimum scores, attendance, and income criteria, ranks eligible students, and resolves ties using predefined rules.

3. python code:

```
students=[]
marks=[]
attendance=[]
income=[]
for i in range(int(input("Enter how many students: "))):
    students.append((input("Enter your name: ")))
    marks.append(int(input("Enter your marks percentage: ")))
    attendance.append(int(input("Enter your attendance percentage: ")))
    income.append(int(input("Enter your income: ")))
eligible=[]
for j in marks:
    if j>60:
        a=marks.index(j)
        eligible.append(students[a])
print("The eligible students are: ")
for k in eligible:
    print(k,end=" ")
    b=students.index(k)
    if income[b]<80000:
        print("with scholarship of 70%")
    elif income[b]>120000:
        print("with scholarship of 50%")
    else:
        print("with scholarship of 30%")
```

4. test cases and outputs:

```
Enter how many students: 2
Enter your name: mustafavali
Enter your marks percentage: 70
Enter your attendance percentage: 75
Enter your income: 70000
Enter your name: karthik
Enter your marks percentage: 80
Enter your attendance percentage: 60
Enter your income: 80000
The eligible students are:
mustafavali with scholarship of 70%
karthik with scholarship of 30%
```